

Contents

S6 — Three Candidate Rerank Signals: One Shipped, Two Rejected by Measurement	1
1. Shipped: negative facet evidence	1
The gap	1
The signal	1
The bug the first version had — and what it taught	2
Measurements (weight 0.0 → 0.4)	2
An honest note on the original hypothesis	2
2. Implemented, measured, removed: profile-weighted facet agreement	2
3. Rejected before implementation: budget/price closeness	3
4. Housekeeping shipped alongside	3
5. Reproduction	3

S6 — Three Candidate Rerank Signals: One Shipped, Two Rejected by Measurement

File changed: `src/rerank.py` (one new signal + housekeeping); supporting edits in `src/facets.py`, `src/policy.py`, `tools/sweep.py`, `tests/test_components.py`.

Result: public set 0.9128 → **0.9143** (200/200 kept), dev split 0.9207 → 0.9224, holdout 0.9010 → 0.9021, adversarial hard set 0.7914 → 0.7917 with no bucket regressing.

This is the record of a three-signal investigation. One signal shipped (negative facet evidence), one was implemented, measured, and removed (profile-weighted agreement), and one was rejected before a line of code was written (budget/price closeness). The negative results are documented with the same care as the positive one — knowing *why* a plausible signal does not work is what stops it being rebuilt later.

All numbers were measured on the current tree in one session, so before/after pairs are directly comparable. (The hard-set baseline here is 0.7914, not the 0.8007 recorded before the anchor-retrieval merge — the merge moved it.)

1. Shipped: negative facet evidence

The gap

Every existing rerank term is positive evidence: span coverage, facet agreement, category and tail alignment. None of them can tell apart a candidate that *satisfies* “color: grey” from one that merely *doesn’t contradict* it — a black-only shirt matches “cotton shirt” spans exactly as well as a grey one and loses nothing for being black. The customer’s constraint rules products out; the scorer only ever ruled products in.

The signal

`_facet_conflicts(customer_facets, product_facets, product_text)` counts each stated facet value the candidate contradicts, and the score subtracts `facet_conflict_weight * conflicts`. A conflict requires all three of:

1. **The customer stated a value** for the attribute (via `extract_query_facets` — material, color, style, use_case, size).
2. **The candidate resolves that attribute too.** Silence is never punished: a product that mentions no colour at all is not in conflict with “grey”. Missing data is not disagreement.
3. **The stated value appears nowhere in the candidate’s text** as a word-bounded substring, aliases included (grey gray is the only synonym pair in the facet vocabulary). This guards against `extract()`’s first-match-wins behaviour: a “black/grey reversible” belt extracts `color: black` but still contains “grey”, and must not be penalised.

The bug the first version had — and what it taught

The first implementation judged conflicts against `full_text()` (everything the customer ever said). On the adversarial override bucket this *punished the target for obeying the intent override*: after “actually, ignore black — I need grey”, the stale “black” was still in the transcript, still extracted first, and grey-only products (the target among them) were penalised for contradicting it. Measured cost: `generic_override` MRR 0.673 $\rightarrow$ 0.626.

The fix judges conflicts against `focused_text()` — the currently authoritative turns only, identical to `full_text()` until an override fires. That restored `generic_override` to exactly 0.673 and is now covered by a unit test (`test_override_discards_stale_facet_for_conflict_scoring`). The general lesson is recorded here deliberately: **negative evidence is more override-sensitive than positive evidence**. A stale positive term merely boosts some wrong candidates; a stale negative term actively demotes the right one.

Measurements (weight 0.0 $\rightarrow$ 0.4)

split	score off	score on	MRR off	MRR on
dev (120)	0.9207	0.9224	0.862	0.870
holdout (80)	0.9010	0.9021	0.811	0.815
public full (200)	0.9128	0.9143	0.842	0.848
hard (96)	0.7914	0.7917	0.695	0.696

Hard set per bucket (MRR): every bucket flat or up, none down — `budget_only_signal` 0.732 $\rightarrow$ 0.736, `homogeneous_cluster` 0.429 $\rightarrow$ 0.431, the other four unchanged. Public hit rate stays 200/200, which was the disqualifying check: a false conflict that demotes a true target would show up there first.

Weight choice: dev scores 0.9224 at 0.4 and 0.9226 at 0.8 — a plateau, and a penalty term gets the smallest weight on it, so 0.4 shipped. At 0.4, one conflict (-0.4) can reorder saturated ties but cannot overturn a genuine span lead (each matched span is worth 1.12).

An honest note on the original hypothesis

This signal was proposed to attack the `homogeneous_cluster` bucket (MRR 0.429), where positive signals saturate. Measurement showed that hypothesis was wrong: bucketmates in a homogeneous cluster share the stated facets by construction, so there is nothing to contradict *within* the cluster and the bucket barely moves ($+0.002$). The gain comes from the general distribution instead — cross-cluster impostors that match the spans but not the stated colour or material. The signal earned its place on the measured gain, not on the story it was designed around.

2. Implemented, measured, removed: profile-weighted facet agreement

`user_profile.preference_tags` (e.g. `["material", "fit"]`) is unused in the scored path. The idea: give extra credit when a facet agreement lands on an attribute the profile says this customer cares about — a personalisation prior, additive-only, mapped through the shared `TAG_HINTS` vocabulary.

config	dev	holdout
baseline (conflict shipped)	0.9224	0.9021
profile_weight 0.1	0.9237	0.9013
profile_weight 0.2	0.9217	0.9005
profile_weight 0.4	0.9217	—

The dev gain at 0.1 does not reproduce on holdout (-0.0008 , and monotonically worse as the weight grows). The shape is a knife-edge, not a plateau — the signature of fitting noise. Removed per the repo convention (dead options are deleted, not kept at weight 0; the pool-local-rarity precedent).

Why it fails is worth keeping: **the profile is redundant with the transcript**. The simulated customer already discloses their profiled preferences verbatim during the session — a customer whose profile says “material” *says the material out loud* by turn 2 or 3, and the facet-agreement term already scores it. Re-weighting that same agreement adds variance without adding information. A profile signal would only pay off where the profile carries information the conversation lacks, which this evaluator’s design never produces.

One durable side effect kept: `TAG_HINTS` moved from `InfoGainPolicy` to `src/facets.py` (module level) with identity entries added, so a literal tag like “material” now maps to the `material` attribute. `InfoGainPolicy` aliases it unchanged; profile tags that name an attribute directly previously did nothing in its answerability prior.

3. Rejected before implementation: budget/price closeness

The idea — long recorded in `docs/team/hard_cases.md` as “budget is triply dead” — was to parse the customer’s “budget around \$X” and score candidates by price closeness, since the disclosed figure is the target’s own price.

Measurement against the evaluator’s own `intent_card()` killed it before any code was written:

population	cards containing a budget constraint
public set (200 sessions)	0
hard set (96 sessions)	4 — all rendered budget around \$- (no number)
full catalog (50,000 intent cards)	224 (0.45%), of which 110 parseable (0.22%)

The root cause is structural: `intent_card()` appends the budget line *after* all feature/detail candidates and keeps only the first four, so a budget surfaces only for near-empty “degenerate card” products. The four hard-set sessions that do carry one are exactly the four **never_retrieved** misses — the target is not in the pool, so no rerank term of any kind can reach them. And their dollar figure renders as “—” anyway.

Expected effect on a private 800 drawn the same way: ~ 2 sessions, likely zero. The signal was not built. If a future evaluator version emits budgets routinely, the design is on file: parse the last-mentioned dollar amount, score 1.0 within $\pm 10\%$ with linear decay to $\pm 50\%$ (never exact-match — robust to rounding), weight ~ 0.6 .

4. Housekeeping shipped alongside

- Deleted the dead commented-out block in `_facet_agreement` (the pre-lowercase `extract()` call kept as a `'''...'''` string).
- `extract_query_facets(state.full_text())` was recomputed for **every candidate** ($300\times$ per turn); it is now computed once per `rerank()` call and passed in. Verified bit-identical before any signal work: public run exactly 0.912801 before and after.
- The scoring formula in `docs/team/ideas.md` was three signals out of date (still showed `span + bm25 + popularity` over depth 200); now matches `RerankConfig`.

5. Reproduction

```
python3 -m unittest discover -s tests           # 51 tests
python3 -m evaluator.local_evaluator           # 0.9143, hit 1.0
```

```
python3 -m evaluator.local_evaluator \
  --dataset data/hard_set.jsonl # 0.7917
python3 tools/sweep.py --split dev \
  --configs conflict00,conflict04,conflict08 # the ablation rows
```

New unit tests (`tests/test_components.py`): `conflict` demotes a contradicting candidate; a multi-value product containing the stated value is not punished; silence about a facet is not a conflict; an override discards the stale value for conflict scoring; weight 0.0 reproduces the previous ranking exactly.